GPIO Driver Documentation

1. Driver Overview

The GPIO driver provides a hardware abstraction layer for the General Purpose Input/Output peripheral of the STM32F401 microcontroller. It allows the application layer to configure and control GPIO pins without directly manipulating hardware registers.

The driver abstracts register-level operations such as clock control, pin configuration, data read/write operations, alternate function configuration, and interrupt handling. This improves code readability, portability, and maintainability.

The driver supports:

- GPIO clock control
- Pin initialization and deinitialization
- Input and output operations
- Alternate function configuration
- External interrupt configuration
- NVIC interrupt configuration and priority management

---

2. Driver Architecture

The application interacts with the GPIO peripheral through a set of APIs exposed by the driver.

Application Layer

↓

GPIO Driver APIs

↓

GPIO Registers (MODER, OTYPER, OSPEEDR, PUPDR, AFR, IDR, ODR)

↓

STM32F401 Hardware

The application only provides configuration parameters through the GPIO_Handle_t structure, while the driver internally configures the required registers.

---

3. GPIO Configuration Process

The GPIO_Init() API is responsible for configuring a GPIO pin.

The configuration sequence is:

1. Enable GPIO peripheral clock.
2. Configure pin mode.
3. Configure output speed.
4. Configure pull-up/pull-down settings.
5. Configure output type.
6. Configure alternate function if required.
7. Configure EXTI and SYSCFG registers if interrupt mode is selected.

The driver automatically updates the required register fields and hides hardware-specific details from the application.

---

4. Interrupt Handling

The driver supports GPIO interrupt generation through the EXTI peripheral.

When interrupt mode is selected:

- EXTI trigger registers (RTSR/FTSR) are configured.
- SYSCFG EXTICR registers map the GPIO port to the corresponding EXTI line.
- EXTI interrupt mask register (IMR) is configured.
- NVIC interrupt enable and priority configuration are handled separately using dedicated APIs.

When an interrupt occurs:

1. EXTI sets the pending bit.
2. NVIC invokes the interrupt service routine.
3. The application calls GPIO_IRQHandling().
4. The driver clears the pending interrupt flag.

This abstraction allows the application to use interrupts without direct EXTI register access.

---

5. Available APIs

a) GPIO_PeriClockControl()

Purpose:
Enable or disable clock for a GPIO peripheral.

Abstraction Provided:
Hides RCC AHB1ENR register manipulation from the application.

Parameters:

- pGPIOx → GPIO peripheral base address
- EnorDi → ENABLE or DISABLE

---

b) GPIO_Init()

Purpose:
Initialize a GPIO pin according to user configuration.

Abstraction Provided:
Configures MODER, OTYPER, OSPEEDR, PUPDR, AFR, EXTI, and SYSCFG registers using a single API.

Parameters:

- pGPIOHandle → Pointer to GPIO handle structure

---

c) GPIO_DeInit()

Purpose:
Reset a GPIO peripheral to its default state.

Abstraction Provided:
Uses RCC reset registers internally and hides reset sequence details.

Parameters:

- pGPIOx → GPIO peripheral base address

---

d) GPIO_ReadFromInputPin()

Purpose:
Read logic level from a specific GPIO pin.

Abstraction Provided:
Returns pin state by internally reading the IDR register.

Returns:

- 0 → Logic Low
- 1 → Logic High

---

e) GPIO_ReadFromInputPort()

Purpose:
Read complete GPIO port state.

Abstraction Provided:
Returns 16-bit value from the GPIO input data register.

Returns:

- 16-bit port value

---

f) GPIO_WriteToOutputPin()

Purpose:
Set or reset a GPIO output pin.

Abstraction Provided:
Modifies the corresponding ODR register bit without exposing register operations.

Parameters:

- pGPIOx → GPIO peripheral base address
- PinNumber → GPIO pin number
- Value → GPIO_PIN_SET or GPIO_PIN_RESET

---

g) GPIO_WriteToOutputPort()

Purpose:
Write data to the entire GPIO port.

Abstraction Provided:
Updates all GPIO output pins through a single API call.

---

h) GPIO_ToggleOutputPin()

Purpose:
Toggle the state of a GPIO output pin.

Abstraction Provided:
Performs bit manipulation internally and hides register access.

---

i) GPIO_IRQInterruptConfig()

Purpose:
Enable or disable a GPIO interrupt in NVIC.

Abstraction Provided:
Handles NVIC ISER and ICER register programming.

Parameters:

- IRQNumber → Interrupt number
- EnorDi → ENABLE or DISABLE

---

j) GPIO_IRQPriorityConfig()

Purpose:
Configure interrupt priority.

Abstraction Provided:
Programs NVIC priority registers and hides priority bit positioning details.

Parameters:

- IRQNumber → Interrupt number

- IRQPriority → Priority value

---

k) GPIO_IRQHandling()

Purpose:
Clear pending GPIO interrupt.

Abstraction Provided:
Handles EXTI pending register access internally.

Parameters:

- PinNumber → GPIO pin number

---

6. Abstraction Provided by the Driver

The GPIO driver hides the following hardware details from the application:

- RCC clock enable registers
- GPIO configuration registers
- Alternate function registers
- EXTI interrupt configuration registers
- SYSCFG EXTI mapping registers
- NVIC interrupt enable registers
- NVIC priority registers

The application only uses high-level APIs and configuration structures while the driver performs all required register manipulations internally.

---

7. Conclusion

The GPIO driver provides a clean abstraction layer over the STM32F401 GPIO peripheral. It simplifies GPIO configuration, pin control, alternate function setup, and interrupt management through a set of reusable APIs. By hiding low-level register operations, the driver improves code modularity, readability, and maintainability while still allowing efficient access to hardware resources.